#19 Otherwise, if there is no sibling with more than `m` elements, we will have to merge the node violating invariants with one of its siblings.
#20 Before we return, we still need to re-compute the bounding envelope for current node.
#21 If it gets here, we are at an internal node and the point has been deleted in `node`'s subtree: check also if `node` now violates the invariant about the minimum number of children.

As you can see, this method is as complicated as `insert` (possibly even more complicated!); thus, similarly to what we had done for `insert`, we have broken down the `delete` method using several helper functions, to keep it leaner and cleaner.

This time, however, we won't describe in detail all of the helper methods: all the methods involving finding something "closest to" a node, like function `findSiblingToMergeTo` in listing 10.15, are heuristics that depend on the definition of "closer" that we adopt; as mentioned when describing how `delete` works, we have a few choices, from "shortest distance" (which is also easy to implement) to lower overlap.

For the sake of space, we need to leave these implementations (including the choice of the proximity function) to the reader: if you refer to the material presented in this and the previous section, you should be able to easily implement the versions using Euclidean distance as a proximity criterion.

Listing 10.16 The findClosestEntryInNodesList method